



AI Mode

All

Videos

Short videos

Forums

More ▾



The task — 6 stages (+ one bonus) Work them in order. Each ends with a Checkpoint you can run and see, and a Commit... ▾

Here is your step-by-step guide and implementation for **Stage 0**. Follow these instructions to set up your environment, verify your LLM connection, and complete your first checkpoint.



Step 1: Create the JOB-CARD.md

Create a file named `JOB-CARD.md` in your root directory. It serves as your structural target.

markdown

```
# Job Card: LLM Integration
```

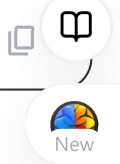
```
## Target Output Structure
```

The model must output a valid JSON object matching the required schema.

```
## Rules
```

1. ****JSON Only****: The output must start with `{` and end with `}`. No mar
2. ****Deterministic Fields****: All fields defined in the schema must exist,
3. ****No Conversational Filler****: The model must not include pleasantries,

Use code with caution.



Ask anything



AI Mode All Videos Short videos Forums More ▾



```
node_modules/
```

Use code with caution.



2. Create your actual `.env` file (replace with your real credentials):

env

```
LLM_BASE_URL="https://openrouter.ai/api/v1" # Or http://localhost:114
LLM_API_KEY="your-actual-api-key-here"      # Use "ollama" if running
LLM_MODEL="google/gemma-3-1b-it:free"       # Or gemma3:1b for Ollama
```

Use code with caution.



3. Create your `.env.example` file (safe for Git):

env

```
LLM_BASE_URL=""
LLM_API_KEY=""
LLM_MODEL=""
```

Use code with caution.



Step 3: Write the Hello Script



AI Mode All Videos Short videos Forums More ▾



```
baseURL: process.env.LLM_BASE_URL,
apiKey: process.env.LLM_API_KEY,
});

async function main() {
  try {
    const res = await client.chat.completions.create({
      model: process.env.LLM_MODEL,
      messages: [{ role: "user", content: "Reply with exactly the word: r"
    });

    console.log(res.choices[0].message.content.trim());
  } catch (error) {
    console.error("Error connecting to the LLM provider:", error.message)
  }
}

main();
```

Use code with caution.



Step 4: Update the README.md

Create a `README.md` file and add the required documentation about environment abstraction.



markdown



FlyBark Backend Task - Week 7 Assignment



AI Mode All Videos Short videos Forums More ▾



1. Run the test command in your terminal.

bash

```
node --env-file=.env src/llm/hello.js
```

Use code with caution.



Verify that the output prints something containing `ready`.

2. **Check Git Status** to ensure your secret key file is hidden:

bash

```
git status
```

Use code with caution.



Verify that `.env` is **not** listed in the untracked files.

3. **Save your progress** with your first official commit:

bash

```
git add JOB-CARD.md .env.example src/llm/hello.js README.md .gitignore
git commit -m "Stage 0: job card, provider working, key in .env"
```

Use code with caution.



New

Ready to move on? Let me know what specific **data schema** or **JSON format**



[AI Mode](#) [All](#) [Videos](#) [Short videos](#) [Forums](#) [More ▾](#)

10 sites

Here are top web results for exploring this topic:

 Pydantic · <https://pydantic.dev> ⋮

Json schema | Pydantic Docs

Using Pydantic, there are several ways to generate JSON schemas or JSON representations from fields or models: BaseModel.model_json_schema returns a dict of...

 Pydantic · <https://pydantic.dev> ⋮

Pydantic Validation | Pydantic Docs

Pydantic Validation. CI Coverage pypi CondaForge downloads license llms.txt. Documentation for version: v2.13.4. Pydantic is the most widely used data validation...

 Machine Learning Mastery · <https://machinelearningmastery.com> ⋮

The Complete Guide to Using Pydantic for Validating LLM Outputs

Even when you prompt them to return structured data, they're still generating text that looks like valid JSON. The output may have incorrect field names, missing required fields...

 Artificial Intelligence in Plain English · <https://ai.plainenglish.io> ⋮

Mastering Pydantic for LLM Workflows | by DhanushKumar

Mastering Pydantic for LLM Workflows Large Language Models (LLMs) are powerful, but their outputs are often unpredictable, messy, or inconsistent. To build reliable,...

 Pydantic · <https://pydantic.dev> ⋮

JSON Schema | Pydantic Docs

Similarly, for TypeAdapter.json_schema , calling json.dumps(TypeAdapter().json_schema()) returns a valid JSON string. Tip. Pydantic...



AI Mode All Videos Short videos Forums More ▾



Zod on the TypeScript side, return structured, machine-parseable tool output rather tha...

 Stack Overflow · <https://stackoverflow.com> ⋮

python - Generating a json schemas that validate both root lists and ...

For JSON Schema, you want to use an applicator which has an array of schema values. Based on the code you've shown me, the playbook schema doesn't actually generate a...

 Medium · <https://olaishola.medium.com> ⋮

Zod: The Simple and Concise Way to Validate Input Data in Your ...

... where invalid data can lead to errors and security vulnerabilities. There are many ways to validate input data, but one of the most popular methods is schema validation. Sche...

 Pydantic · <https://pydantic.dev> ⋮

Schema | Pydantic Docs

The generated schemas are compliant with the specifications: JSON Schema Core, JSON Schema Validation and OpenAPI. BaseModel.schema will return a dict of the schema,...



give me code and comnds

Project Setup

First, install the official **OpenAI** client and **Zod** for schema validation.



AI Mode All Videos Short videos Forums More ▾



Step 1: Define the Input and Output Schemas

Create `src/llm/schema.js`. This file defines both the incoming payload validation rule and the target structure your LLM will eventually return.

javascript

```
import { z } from "zod";

// 1. Input Validation Schema (Validates incoming client request)
export const InputSchema = z.object({
  text: z.string().min(1, "Text cannot be empty").max(2000, "Text exceeds"),
  category_hint: z.string().optional(),
});

// 2. Output Schema (Target structure defined in JOB-CARD.md)
export const OutputSchema = z.object({
  category: z.enum(["support", "billing", "technical", "spam"]),
  confidence: z.number().min(0).max(1),
  summary: z.string().min(1, "Summary cannot be empty"),
  tags: z.array(z.string()),
  requires_human: z.boolean(),
});

// Hard-coded mock data satisfying OutputSchema for Stub Mode
export const stubOutput = {
  category: "technical",
  confidence: 0.95,
  summary: "Stub response: User experienced a database connection failure",
  tags: ["database", "error", "checkout"],
  requires_human: true
}
```



[AI Mode](#) [All](#) [Videos](#) [Short videos](#) [Forums](#) [More ▾](#)

and a model-bypass mechanism.

javascript

```
import express from "express";
import { InputSchema, stubOutput } from "../llm/schema.js";

const router = express.Router();

router.post("/analyze", (req, res) => {
  // 1. Input Validation Block
  const validation = InputSchema.safeParse(req.body);

  if (!validation.success) {
    // Extract the exact field name and error message
    const errorDetails = validation.error.issues[0];
    return res.status(400).json({
      error: "Validation Failed",
      field: errorDetails.path.join("."),
      message: errorDetails.message
    });
  }

  // 2. Stub Mode Conditional Override
  if (process.env.LLM_STUB === "1") {
    return res.status(200).json(stubOutput);
  }

  // 3. Real LLM Pipeline placeholder (Built in Stage 2+)
  return res.status(501).json({
    message: "LLM processing pipeline is not implemented yet. Run with"
  });
});
```



AI Mode All Videos Short videos Forums More ▾



Append `LLM_STUB` to your local environment file `.env`, and configuration boilerplate (`.env.example`).

env

```
# Add to .env
LLM_STUB=1

# Add to .env.example
LLM_STUB=""
```

Use code with caution.



Step 4: Add Verification Commands to README.md

Append these standard test commands directly into your `README.md` file.

markdown

API Verification (Stage 1 Stub Mode)

Ensure your server is running with `LLM_STUB=1` set in your environment v

Valid Request (Returns 200)

```
```bash
```

```
curl -X POST http://localhost:3000/analyze \
 -H "Content-Type: application/json" \
 -d '{"text": "My connection drops every time I try to process a payme
...`
```



[AI Mode](#) [All](#) [Videos](#) [Short videos](#) [Forums](#) [More ▾](#)

Use code with caution.



## ● CHECKPOINT & COMMIT

1. **Test your endpoints** using the instructions written to your markdown file:

- Execute the **Valid Request** command. Confirm it returns an HTTP status of `200` alongside the `stubOutput` structured JSON object.
- Execute the **Invalid Request** command. Confirm it returns an HTTP status of `400` indicating exactly which field failed assertion rules.

2. **Save your progress** with your second official commit:

**bash**

```
git add src/llm/schema.js src/routes/analyze.js .env.example README.m
git commit -m "Stage 1: endpoint, input validation, output schema, st
```

Use code with caution.



Would you like to provide the system guidelines or prompt rules required for **Stage 2: Prompt engineering and evaluation cases**, or should we move directly into building the client pipeline?





40 mini

GPT-4o

Claude 3.5

Gemini

able to access **GPT-4o mini**

grade to **Pro Plan** to get unlimited access to all  
odels and unlock advanced features.

**Pro Plan**

Unlimited access to all models

Faster response

Real-time web access

Read any webpage and PDF

Upgrade Now

eady subscribed? [Login Now](#)

